Documento de Arquitetura – Vehicle Resale API

Projeto	Vehicle Resale API
Versão	1.0
Disciplina	Fase 5 – Dados e Segurança da Informação
Curso	Pós-Graduação FIAP – Arquitetura de Software
Stack	Quarkus 3.6.4 · Java 17 · PostgreSQL 15 · Keycloak 23 · Docker · Kubernetes
Data	Março/2026

Sumário

1. Visão Geral
2. Arquitetura de Componentes
3. Fluxo SAGA – Processo de Compra
4. Modelo de Dados
5. Arquitetura de Implantação em Nuvem (AWS)
6. Serviços de Nuvem Selecionados e Justificativas
7. Segurança em Profundidade
8. Decisões Arquiteturais (ADR)
9. Resumo dos Entregáveis

---

1. Visão Geral

A **Vehicle Resale API** é uma API RESTful construída com **Quarkus 3.6.4 (Java 17)** que gerencia o ciclo de vida completo de revenda de veículos: cadastro de veículos, cadastro de clientes/compradores, criação de vendas com geração de código de pagamento e atualização de status via webhook.

A autenticação e autorização são delegadas ao **Keycloak 23**, implantado como serviço de identidade separado, com banco de dados próprio e isolado.

A arquitetura segue os princípios de **Clean Architecture (Arquitetura Hexagonal)** combinados com **Domain-Driven Design (DDD)**, separando claramente as camadas de domínio, aplicação e infraestrutura.

---

2. Arquitetura de Componentes

2.1 Visão macro — atores e serviços

Os atores externos interagem com a API exclusivamente pelo **Load Balancer** (ALB), que realiza a terminação TLS e aplica regras do WAF antes de encaminhar as requisições.

Ator / Sistema	Interação principal
Cliente anônimo	Consulta veículos disponíveis e vendidos (endpoints públicos)
Comprador autenticado	Cadastra-se como cliente e efetua compras (token Keycloak — role buyer)
Sistema de pagamento	Envia resultado do pagamento via webhook POST
Administrador	Cadastra e gerencia o estoque de veículos (token Keycloak — role admin)

Fluxo de entrada de requisições:

1. Requisição chega ao **ALB** (HTTPS, porta 443) com terminação TLS via ACM.
2. O ALB aplica regras do **AWS WAF** (OWASP Top 10, rate limiting).
3. A requisição é encaminhada ao pod da **Vehicle Resale API** no EKS.
4. A API valida o token JWT junto ao **Keycloak** (OIDC discovery).
5. A política de autorização por path/role é aplicada.
6. A requisição acessa o **PostgreSQL da API** (subnets privadas).

Serviços implantados:

Serviço	Porta	Rede
Vehicle Resale API (Quarkus)	8080 (interno)	Subnet privada — EKS
Keycloak 23 (Identity Provider)	8180	Subnet privada — EKS
PostgreSQL da API	5432	Subnet privada — RDS
PostgreSQL do Keycloak	5432	Subnet privada — RDS
ALB + WAF	443 (público)	Subnet pública

2.2 Camadas da Clean Architecture

A aplicação é estruturada em três camadas, com dependências fluindo de fora para dentro (infraestrutura → aplicação → domínio).

Camada de Infraestrutura — adaptadores externos:

- REST Resources: `VehicleResource`, `CustomerResource`, `SaleResource`, `WebhookResource`
- Persistência: `VehicleRepositoryEnhanced`, `CustomerRepository`, `SaleRepository` (Hibernate/Panache)

Camada de Aplicação — orquestração de casos de uso:

- Controllers: `VehicleController`, `CustomerController`, `SaleController`
- Responsáveis por orquestrar Gateways e Presenters
- Contém a lógica de compensação SAGA

Camada de Domínio — núcleo independente de framework:

Sublayer	Classes principais
Entidades	`Vehicle`, `Customer`, `Sale`
Serviços	`VehicleService`, `CustomerService`, `SaleService`
Gateways	`VehicleGateway`, `CustomerGateway`, `SaleGateway`
Enumerações	`VehicleStatus` (AVAILABLE, SOLD), `PaymentStatus` (PENDING, APPROVED, REJECTED)

---

3. Fluxo SAGA – Processo de Compra

O processo de compra altera o estado de dois recursos (veículo e venda), exigindo um mecanismo de compensação em caso de falha no pagamento.

3.1 Caminho feliz — pagamento aprovado

Etapa	Ação	Resultado
1	`POST /api/sales` recebe dados do comprador e veículo	—
2	`SaleService` valida CPF do comprador no cadastro	CPF encontrado
3	`VehicleController.markAsSold()` é chamado	Veículo → SOLD
4	Venda é persistida com status PENDING e UUID gerado	paymentCode retornado
5	Comprador efetua o pagamento externamente	—
6	Webhook `POST /api/webhook/payment` com `paid: true`	Venda → APPROVED

3.2 Caminho de falha — pagamento rejeitado (compensação SAGA)

Etapa	Ação	Resultado
1–4	Idêntico ao caminho feliz	Veículo SOLD, venda PENDING
5	Webhook `POST /api/webhook/payment` com `paid: false`	Venda → REJECTED
6	**Compensação SAGA:** `VehicleController.markAsAvailable()`	Veículo → AVAILABLE

O registro de venda com status REJECTED é mantido para auditoria. O veículo retorna ao estoque disponível de forma automática e imediata.

3.3 Resumo dos estados do veículo

Momento	Status
Antes da venda	AVAILABLE
Após criação da venda	SOLD
Após pagamento aprovado	SOLD (definitivo)
Após pagamento rejeitado	AVAILABLE (compensado pela SAGA)

---

4. Modelo de Dados

O banco transacional `vehicle_resale_db` (PostgreSQL 15) contém três entidades principais com o seguinte relacionamento: `vehicles` ← `sales` → `customers`.

Entidade `vehicles`

Campo	Tipo	Observação
id (PK)	Long	Identificador único
brand, model, color	String	Dados do veículo
year	Integer	Ano de fabricação
price	BigDecimal	Preço de venda
status	Enum	AVAILABLE ou SOLD
created_at, updated_at, deleted_at	Timestamp	Ciclo de vida

Entidade `customers`

Campo	Tipo	Dado sensível
id (PK)	Long	Não
user_id (FK Keycloak)	String	Não
name	String	Sim
email (unique)	String	Sim
cpf (unique)	String	Sim
phone	String	Sim
address, city, state, zip_code	String	Sim
active	Boolean	Não
created_at, updated_at	Timestamp	Não

Entidade `sales`

Campo	Tipo	Dado sensível
id (PK)	Long	Não
vehicle_id (FK)	Long	Não
buyer_name	String	Sim
buyer_email	String	Sim
buyer_cpf	String	Sim
sale_date, sale_price	Date / Decimal	Não
payment_code (UUID)	String	Operacional
payment_status	Enum	Não
created_at, updated_at	Timestamp	Não

O banco do Keycloak (`keycloak_db`) é gerenciado exclusivamente pelo serviço de identidade e **não armazena** CPF, endereço nem dados financeiros da API.

---

5. Arquitetura de Implantação em Nuvem (AWS)

5.1 Justificativa da escolha da AWS

A AWS foi escolhida como plataforma de nuvem pelos seguintes motivos:

- **Maturidade e ecossistema:** serviços gerenciados para todos os componentes da solução (EKS, RDS, Secrets Manager, WAF, ACM).
- **Serviços serverless nativos:** suporte a contêineres via Fargate sem necessidade de gerenciar servidores EC2.
- **Conformidade com LGPD:** região brasileira `sa-east-1` (São Paulo) com certificações SOC 2, ISO 27001 e PCI DSS; contratos DPA disponíveis.
- **Segurança gerenciada:** WAF, Shield, GuardDuty, Secrets Manager e KMS integrados de forma nativa.

5.2 Topologia de rede na AWS

Região: `sa-east-1` (São Paulo) — VPC `10.0.0.0/16`

Subnets públicas (`10.0.1.0/24`):

- Application Load Balancer (ALB) com AWS WAF integrado

- Terminação TLS via AWS Certificate Manager (ACM)
- Amazon Route 53 para resolução de DNS

Subnets privadas (`10.0.2.0/24`):

- Amazon EKS Cluster com AWS Fargate
  - Pod: `vehicle-resale-api` (Quarkus)
  - Pod: `keycloak-idp` (Keycloak 23)
- Amazon RDS PostgreSQL Multi-AZ
  - Instância: `vehicle_resale_db`
  - Instância: `keycloak_db` (separada)
- AWS Secrets Manager
  - Senhas de banco, client_secret do Keycloak, chave do webhook

Serviços transversais de segurança e observabilidade:

- AWS WAF, AWS Shield Standard, AWS GuardDuty, AWS CloudTrail
- Amazon CloudWatch, AWS KMS, AWS ACM, Amazon ECR

---

6. Serviços de Nuvem Selecionados e Justificativas

6.1 Computação

Serviço	Função
Amazon EKS	Orquestração de contêineres (API + Keycloak). Gerenciado, sem controle do control plane; escalonamento automático; integração nativa com ALB, IAM e Secrets Manager.
AWS Fargate	Execução serverless dos Pods sem gerenciar nós EC2. Billing por CPU/memória consumidos; ideal para workloads variáveis.
Amazon ECR	Registro de imagens Docker. Integrado ao pipeline CI/CD (GitHub Actions → ECR → EKS); suporte a scan de vulnerabilidades.

6.2 Banco de Dados

Serviço	Função
Amazon RDS PostgreSQL Multi-AZ	Banco transacional gerenciado. Backups automáticos, failover Multi-AZ, patches gerenciados, criptografia em repouso via KMS. Duas instâncias isoladas: `vehicle_resale_db` e `keycloak_db`.
Amazon RDS Proxy (opcional)	Pool de conexões entre EKS e RDS. Reduz latência, suporta failover transparente e integração com IAM Authentication.

6.3 Rede e Exposição

Serviço	Função
Amazon VPC	Isolamento de rede. Subnets públicas (ALB) e privadas (EKS, RDS); Security Groups restringem tráfego.
Application Load Balancer (ALB)	Entrada pública com terminação TLS. Integração com EKS Ingress Controller, ACM e health checks.
AWS Certificate Manager (ACM)	Certificados TLS gerenciados. Provisionamento e renovação automáticos sem custo adicional.
Amazon Route 53	DNS com roteamento de domínio e health checks de failover.

6.4 Segurança

Serviço	Função
AWS WAF	Firewall de aplicação integrado ao ALB. Protege contra OWASP Top 10; rate limiting por IP; regras gerenciadas pela AWS.
AWS Shield Standard	Proteção automática contra ataques DDoS de camada 3/4, incluso sem custo extra.
AWS Secrets Manager	Armazenamento de credenciais. Rotação automática; integração com EKS via External Secrets Operator.
AWS KMS	Criptografia em repouso. Chaves gerenciadas para RDS e, opcionalmente, para CPF em nível de aplicação.
AWS GuardDuty	Detecção de ameaças e comportamentos anômalos. Monitora CloudTrail, VPC Flow Logs e DNS.
AWS CloudTrail	Auditoria de todas as ações na conta AWS. Trilha de conformidade para LGPD e SOC 2.
AWS IAM	Controle de acesso a serviços AWS pelo princípio do menor privilégio. Roles para EKS nodes e Fargate tasks.

6.5 CI/CD e Observabilidade

Serviço	Função
GitHub Actions	Pipeline CI/CD já configurado no projeto. Build, testes, push para ECR e deploy no EKS.
Amazon CloudWatch	Logs centralizados dos Pods EKS via Fluent Bit; alertas e integração com Grafana.
AWS X-Ray (opcional)	Rastreamento distribuído para cenários com múltiplas APIs ou microsserviços.

---

7. Segurança em Profundidade

A solução adota um modelo de segurança em cinco camadas, do perímetro de rede até o monitoramento contínuo.

Camada 1 — Rede

- VPC com Security Groups e NACLs segmentando o tráfego
- ALB com WAF aplicando regras OWASP, rate limiting e bloqueio de IPs
- AWS Shield Standard para proteção DDoS
- Acesso externo apenas via ALB nas portas 80/443
- EKS Nodes e RDS em subnets privadas, sem IP público

Camada 2 — Identidade e Acesso

- Keycloak 23 (OAuth2/OIDC) para autenticação de usuários
- Roles: `admin` (gestão de veículos) e `buyer` (comprar e consultar)
- AWS IAM para acesso a serviços AWS (sem access keys em código)
- Credenciais armazenadas no AWS Secrets Manager

Camada 3 — Aplicação

- Autenticação obrigatória por path configurada no Quarkus
- Validação de entrada (Bean Validation) em todos os DTOs
- HTTPS forçado com HSTS (TLS terminado no ALB)
- Webhook de pagamento: recomendado header secreto ou assinatura HMAC

Camada 4 — Dados

- RDS criptografado em repouso via AWS KMS
- Dados sensíveis (CPF, e-mail, endereço) acessíveis apenas por usuários autenticados
- Recomendação: mascaramento de CPF nos logs de aplicação
- Recomendação: criptografia AES-256 em nível de aplicação para CPF
- Política de retenção e exclusão alinhada à LGPD

Camada 5 — Auditoria e Monitoramento

- AWS CloudTrail para auditoria de toda ação na infraestrutura
- AWS GuardDuty para detecção de ameaças e anomalias
- Amazon CloudWatch Logs para logs centralizados da aplicação e do Keycloak
- Prometheus + Grafana para métricas e alertas operacionais

7.1 Conformidade LGPD

Princípio (Art. 6º)	Implementação na solução
Finalidade	CPF e endereço coletados para pagamento e documentação na retirada
Necessidade	Apenas dados estritamente necessários (sem biometria, sem documentos além do CPF)
Acesso do titular	`GET /api/customers/{id}` disponível para o próprio titular (autenticado)
Segurança	Auth Keycloak, autorização por role, TLS em trânsito, KMS em repouso
Responsabilização	CloudTrail e logs auditáveis; DPA com a AWS disponível para `sa-east-1`

---

8. Decisões Arquiteturais (ADR)

ADR-01 — Clean Architecture (Hexagonal)

Decisão: adotar Clean Architecture com separação em camadas Domain, Application e Infrastructure.

Justificativa: desacopla a lógica de negócio de frameworks e detalhes de infraestrutura; facilita testes unitários; permite troca de banco ou framework sem alterar o domínio.

Consequências: mais arquivos e abstrações; necessidade de mappers DTO ↔ Entity; compensado pela testabilidade e manutenibilidade.

ADR-02 — Keycloak como serviço separado

Decisão: Keycloak em Pod separado com banco próprio (`keycloak_db`).

Justificativa: separação entre identidade e dados de negócio; banco de identidade isolado reduz superfície de ataque; Keycloak escalável independentemente.

Consequências: dois bancos PostgreSQL a gerenciar; configuração OIDC necessária; compensada pela robustez de segurança.

ADR-03 — SAGA de Compensação para rejeição de pagamento

Decisão: implementar SAGA de compensação dentro do próprio serviço (`SaleService` + `VehicleController.markAsAvailable()`).

Justificativa: fluxo com dois recursos e único ponto de falha (pagamento rejeitado); compensação local é suficiente e evita complexidade de um orquestrador externo.

Consequências: se o sistema evoluir com mais serviços (gateway de pagamento externo, notificações), será necessário migrar para SAGA orquestrada com motor de estados, como AWS Step Functions.

ADR-04 — Amazon EKS com Fargate

Decisão: usar EKS com Fargate para execução serverless dos Pods.

Justificativa: elimina gestão de servidores EC2; billing baseado em consumo real; integração nativa com IAM, Secrets Manager e ALB.

Consequências: limitações do Fargate (sem DaemonSets, sem SSH a nodes); aceitas dado o contexto do projeto.

ADR-05 — Amazon RDS PostgreSQL Multi-AZ

Decisão: usar RDS PostgreSQL gerenciado em vez de PostgreSQL self-managed em contêiner.

Justificativa: backups automáticos; failover Multi-AZ (SLA 99,95%); patches de segurança gerenciados; criptografia em repouso inclusa.

Consequências: custo maior que contêiner self-managed; aceito dado o SLA e a criticidade dos dados.

---

9. Resumo dos Entregáveis

Entregável	Arquivo	Situação
Desenho da Arquitetura	`docs/ARQUITETURA_FASE5.md`	Entregue
Relatório de Segurança de Dados	`docs/RELATORIO_SEGURANCA_DADOS_FASE5.md`	Entregue
Relatório de Orquestração SAGA	`docs/RELATORIO_SAGA_FASE5.md`	Entregue
Validação dos Requisitos	`docs/FASE_5_VALIDACAO_REQUISITOS.md`	Entregue
Código-fonte	Repositório `vehicle-resale-api` (GitHub)	Disponível

Documento elaborado para a Fase 5 – Dados e Segurança da Informação – Pós-Graduação FIAP – Arquitetura de Software – Março/2026.